FöhnCast

Personalized Kiteboarding Forecast for Central Switzerland

MLOPS_FS26 · Project Proposal · Milestone 1

Author Javier Izquierdo **Supervisor** Dr. Marc Bravin

Programme BSc Artificial Intelligence, HSLU

Repository `https://github.com/javihslu/foehncast`

Project board `https://github.com/users/javihslu/projects/8`

1 Problem Statement

Kiteboarding in Central Switzerland depends on highly localized wind patterns — particularly the *föhn*, a warm downslope wind that can produce excellent conditions at one lake while a spot 30 km away remains flat. Riders currently rely on raw weather apps and community chat groups, often wasting hours driving to the wrong location.

FöhnCast addresses this by predicting a **personalized quality index (0–5)** for every configured Swiss kiteboarding spot, then recommending the best options via a weighted ranking: forecast quality (60 %), ride-to-drive-time ratio (30 %), and forecast window duration (10 %). The system is **config-driven** — spots are defined in `config.yaml` with coordinates, shore orientation, and ideal wind direction. The initial release ships with six validated lake spots; adding new locations requires only a YAML entry. Predictions are personalized to the rider's body weight, kite quiver, and home location (default: 80 kg, Schwyz, 5 kites from 5–12 m^2).

Research question: Can a lightweight, fully automated MLOps pipeline turn public weather forecasts into reliable, personalized kiteboarding recommendations — deployable locally via Docker Compose and optionally on GCP Cloud Run?

2 Data Sources

All external data is free, requires no authentication, and is fetched at runtime via HTTP. Endpoints and parameters are defined in `config.yaml` so they can be changed without touching code.

Source	Access	Format	Role in Pipeline
Open-Meteo Forecast	REST API	JSON	Hourly 7-day forecasts: wind at 10/80/120 m, gusts, temperature, humidity, cloud cover, precipitation, pressure, CAPE, lifted index (13 params).
Open-Meteo Archive	REST API	JSON	Multi-year historical weather for label generation and offline model training.
MeteoSwiss	Open data	JSON	10-minute ground wind observations from 160 SwissMetNet stations for forecast sanity checks.
OSRM	REST API	JSON	Real-time drive-time calculation from rider home to each spot (personalizes ranking).
`config.yaml`	Local file	YAML	Spot definitions (coordinates, shore orientation, ideal wind), rider profile, model parameters.

3 Features and Labels

Open-Meteo delivers **13 raw parameters** per spot and hour: wind speed at three heights (10/80/120 m), direction at two (10/80 m), gusts at 10 m, temperature, precipitation, humidity, cloud cover, sea-level pressure, CAPE, and lifted index. The feature pipeline engineers four derived columns, then selects a **9-feature model vector** (6 raw + 3 engineered):

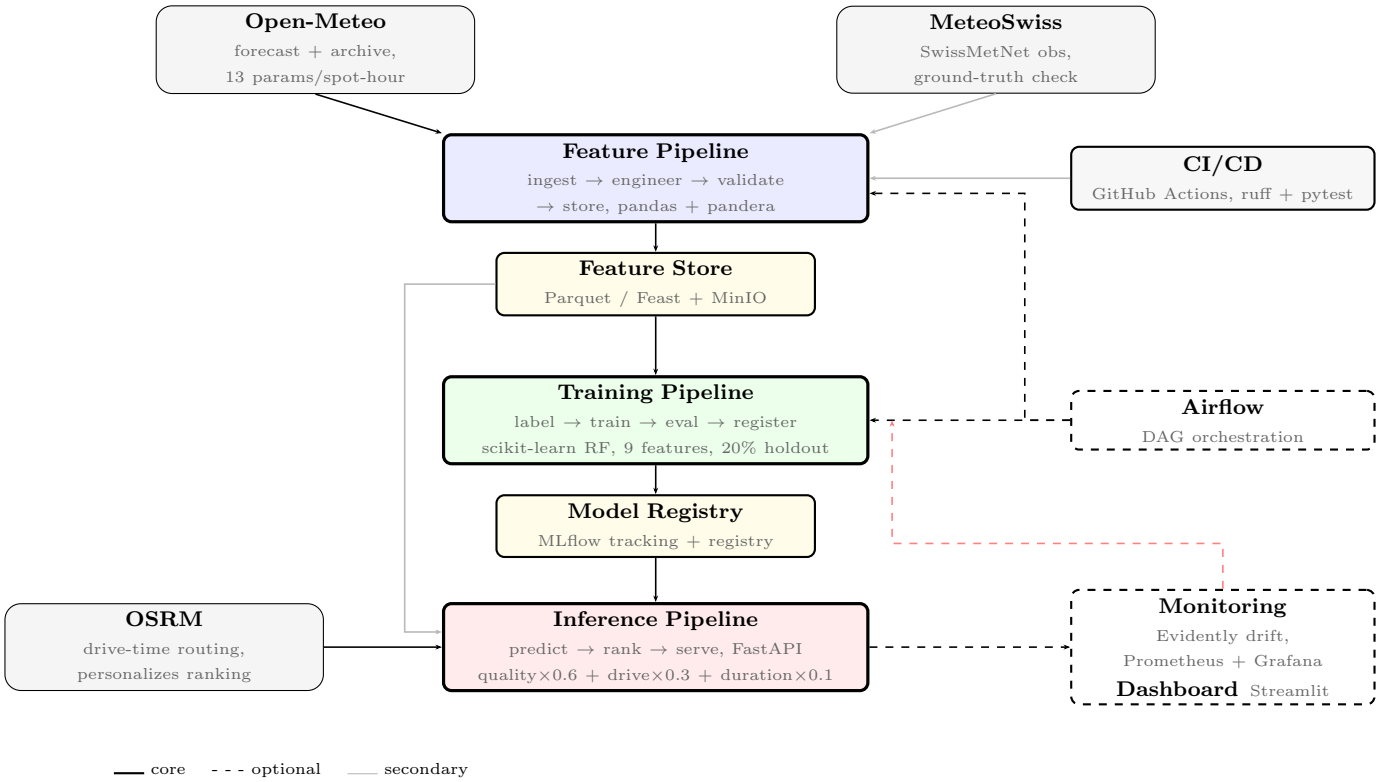
Feature	Type	Definition
`wind_speed_10m, _80m`	raw	Sustained wind at hub heights relevant to kiting.
`wind_direction_10m`	raw	Wind direction relative to shore orientation.
`wind_gusts_10m`	raw	Peak gust speed; safety and comfort indicator.
`temperature_2m`	raw	Air temperature; affects rider endurance.
`relative_humidity_2m`	raw	Humidity; correlated with precipitation risk.
`wind_steadiness`	eng.	Coefficient of variation of 10 m wind over 3 h. Low = consistent, kiteable conditions.
`gust_factor`	eng.	Gust-to-sustained ratio. High = dangerous.
`shore_alignment`	eng.	Cosine similarity of wind direction to spot's ideal shore orientation. Cross-shore scores highest.
`kite_size_optimal`	eng.	Whether the rider's quiver covers the wind range. *(computed; reserved for v2.)*

Training labels follow a rule-based **quality index (0–5)**. Level 5 is not the strongest wind but the best *overall* conditions (favorable direction, low gust ratio, cross-shore flow):

	Label	Wind (kts)	Meaning
0	Dangerous	>40, gusts >50	Unsafe storm. Stay home.
1	Waste of Time	<12 / <15	Too light for even the largest kite.
2	Marginal	12–17	Light-wind session on biggest kite.
3	Good Enough	17–22	Solid session, mid-range kite.
4	Fun Day	22–30	Strong, consistent. Ideal conditions.
5	Perfect Storm	15–25, cross-shore	Optimal direction & steadiness.

4 Architecture — FTI Pipelines

FöhnCast implements the **Feature–Training–Inference (FTI)** pattern. Each pipeline is an independent Python module in `src/foehncast/`; they communicate only through two shared services: a **feature store** (Parquet files *or* Feast + MinIO) and a **model registry** (MLflow). The diagram below shows the top-to-bottom data flow; dashed boxes are optional enhancements.



Each pipeline maps to a sub-package in `src/foehncast/`; an optional GCP Cloud Run deployment (europe-west6) mirrors the local Docker Compose stack.

5 Tech Stack

Layer	Technology & Rationale
Language	Python 3.12, `uv` package manager, hatchling build backend
Data	pandas → Parquet (columnar, zero-cost local store); *opt:* Feast feature store with MinIO S3 backend
Validation	pandera schema contracts on every ingest; *opt:* DVC for data versioning
Model	scikit-learn Random Forest (interpretable, fast on tabular data); MLflow experiment tracking + model registry
Serving	FastAPI REST API (async, auto-generated OpenAPI docs); *opt:* Streamlit dashboard
Orchestration	*opt:* Airflow DAGs — thin wrappers calling pipeline modules
CI/CD	GitHub Actions: lint (ruff), test (pytest), containerize; pre-commit hooks for local checks
Monitoring	*opt:* Evidently AI for drift detection; Prometheus + Grafana for service metrics
Infra	Docker Compose with two-network isolation (frontend / backend); *opt:* GCP Cloud Run (europe-west6)

Italic = optional enhancements (nice to have). The core system is fully functional without them.